

Assignment : 20.2

Name: B.shashank

Htno: 2303A51431

Batch: 21

Task 1 – Password Security Enhancement

Task:

Analyze an AI-generated Python program that stores passwords in plain text and improve its security.

Instructions:

- Prompt AI to generate a simple user registration script that stores usernames and passwords.
- Check whether passwords are stored as plain text.
- Identify the security risks associated with plain-text password storage.
- Refactor the program to use secure password hashing (e.g., hashlib or bcrypt).
- Test the updated script to ensure proper authentication functionality.

Expected Output:

A secure Python script that stores hashed passwords instead of plain text and successfully validates user login credentials.

Prompt:

Generate a simple Python user registration and login system that stores usernames and passwords.

Code:

```

1  """
2  Generate a simple Python user registration and login system
3  that stores usernames and hashed passwords.
4  """
5
6  import hashlib
7
8  # Function to hash password
9  def hash_password(password):
10     return hashlib.sha256(password.encode()).hexdigest()
11
12  # Function to register user
13  def register_user(username, password, user_dict):
14     if username in user_dict:
15         print("Username already exists.")
16         return False
17     user_dict[username] = hash_password(password)
18     print("User registered successfully.")
19     return True
20
21  # Function to login user
22  def login_user(username, password, user_dict):
23     if username not in user_dict:
24         print("Invalid username or password.")
25         return False
26
27     if user_dict[username] == hash_password(password):
28         print("Login successful.")
29         return True
30     else:
31         print("Invalid username or password.")
32         return False
33
34  # Main function
35  def main():

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```

Do you want to register (r) or login (l)? (q to quit): r
Enter username: shashank
Enter password: shashank@93810
User registered successfully.
Do you want to register (r) or login (l)? (q to quit): l
Enter username: shashank
Enter password: shashank@93810
Login successful.

```

Explanation:

- The AI-generated code stored passwords in **plain text**.
- This is dangerous because:
 - Hackers can easily read passwords if data is leaked.
 - No protection for user credentials.

Task 2 – Secure API Key Management

Task:

Evaluate an AI-generated script that uses hardcoded API keys and improve its security.

Instructions:

- Ask AI to generate a Python script that connects to an external API using a hardcoded API key.
- Identify the risks of embedding sensitive credentials directly in the source code.
- Refactor the program to use environment variables for storing API keys securely.
- Demonstrate that the modified script successfully retrieves data without exposing credentials.

Expected Output:

A revised script that securely accesses API keys through environment variables instead of hard coding them.

Prompt:

Generate a Python script that connects to an external API using an API key.

'''Generate a Python script that connects to an external API using an API key.'''

```
import requests
```

```
API_KEY = "your api key here"
```

```
API_URL = "https://api.example.com/data"
```

```
headers = {
```

```
    "Authorization": f"Bearer {API_KEY}"
```

```
}
```

```
response = requests.get(API_URL, headers=headers)
```

```
print(response.json())
```

```
Enter your choice: 3
```

```
Fetching API data securely...
```

```
Data fetched successfully!
```

```
ID: 1
```

```
Title: sunt aut facere repellat provident occaecati excepturi optio reprehenderit
```

```
Body: quia et suscipit
```

```
suscipit recusandae consequuntur expedita et cum  
reprehenderit molestiae ut ut quas totam  
nostrum rerum est autem sunt rem eveniet architecto
```

```
-----
```

```
ID: 2
```

```
Title: qui est esse
```

```
Body: est rerum tempore vitae
```

```
sequi sint nihil reprehenderit dolor beatae ea dolores neque  
fugiat blanditiis voluptate porro vel nihil molestiae ut reiciendis
```

```
qui aperiam non debitis possimus qui neque nisi nulla
```

```
-----
```

Explanation:

- Sensitive data is not exposed in source code.
- Improves security and prevents misuse.
- Code becomes safe for sharing and deployment.

Task 3 – Weak Encryption Detection

Task:

Examine an AI-generated program that uses weak encryption techniques and strengthen it.

Instructions:

- Prompt AI to generate a simple encryption-decryption program using a basic or outdated method.
- Identify weaknesses in the encryption approach (e.g., reversible encoding or outdated algorithms).

- Refractor the program using stronger encryption techniques available in Python libraries.
- Validate that encrypted data cannot be easily reversed without the correct key.

Expected Output:

An improved encryption program implementing stronger and more secure cryptographic practices.

Prompt:

Generate a simple Python encryption and decryption program.

```
.vscode > 1.py > ...
1  '''Generate a simple Python encryption and decryption program.'''
2  def encrypt(text, shift):
3      '''Encrypt the text using a simple Caesar cipher.'''
4      result = ""
5      for char in text:
6          if char.isalpha():
7              shift_amount = shift % 26
8              if char.islower():
9                  result += chr((ord(char) - ord('a') + shift_amount) % 26 + ord('a'))
10             else:
11                 result += chr((ord(char) - ord('A') + shift_amount) % 26 + ord('A'))
12             else:
13                 result += char
14         return result
15
16 def decrypt(text, shift):
17     '''Decrypt the text using a simple Caesar cipher.'''
18     return encrypt(text, -shift)
19 # Example usage
20 if __name__ == "__main__":
21     original_text = "Hello, World!"
22     shift = 3
23     encrypted_text = encrypt(original_text, shift)
24     decrypted_text = decrypt(encrypted_text, shift)
25     print(f"Original text: {original_text}")
26     print(f"Encrypted text: {encrypted_text}")
27     print(f"Decrypted text: {decrypted_text}")
28
```

PROBLEMS 7 OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\2303A\OneDrive\Desktop\aiac> & C:\Users\2303A\AppData\Local\Programs\Python\Python313\top\aiac/.vscode/1.py
Original text: Hello, World!
Encrypted text: Khoor, Zruog!
Decrypted text: Hello, World!
PS C:\Users\2303A\OneDrive\Desktop\aiac>
```

Explanation:

- Data is securely encrypted using a key.

- Without the key, decryption is not possible.
- Ensures confidentiality of sensitive data.

Task 4 – Real-Time Application: Security Review of AI-Generated Web Service

Scenario:

Students select an AI-generated web service snippet (e.g., REST API, file upload handler, or authentication module).

Instructions:

- Perform a structured security review of the generated code.
- Identify at least three potential vulnerabilities.
- Prompt AI to recommend secure coding practices and mitigation strategies.
- Implement the suggested improvements and test the secured version.
- Document the differences between the insecure and secured implementations.

Expected Output:

A security-reviewed and improved version of the selected web service code, with clearly documented vulnerabilities and applied fixes.

Prompt:

Generate a simple Flask API with login functionality.

Code:


```

.vscode > 1.py > ...
1  '''Generate a simple Flask API with login functionality. '''
2  from flask import Flask, request, jsonify
3  from werkzeug.security import generate_password_hash, check_password_hash
4  app = Flask(__name__)
5  # In-memory user storage
6  users = {}
7  @app.route('/register', methods=['POST'])
8  def register():
9      data = request.get_json()
10     username = data.get('username')
11     password = data.get('password')
12     if username in users:
13         return jsonify({'message': 'User already exists'}), 400
14     users[username] = generate_password_hash(password)
15     return jsonify({'message': 'User registered successfully'}), 201
16 @app.route('/login', methods=['POST'])
17 def login():
18     data = request.get_json()
19     username = data.get('username')
20     password = data.get('password')
21     if username not in users or not check_password_hash(users[username], password):
22         return jsonify({'message': 'Invalid credentials'}), 401
23     return jsonify({'message': 'Login successful'}), 200
24 if __name__ == '__main__':
25     app.run(debug=True)
26

```

PROBLEMS 6 OUTPUT DEBUG CONSOLE TERMINAL PORTS

Decrypted text: Hello, World!

PS C:\Users\2303A\OneDrive\Desktop\aiac> & C:\Users\2303A\AppData\Local\Programs\Python\Python313\python top\aiac/.vscode/1.py

* Serving Flask app '1'

* Debug mode: on

WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI s

* Running on http://127.0.0.1:5000

Press CTRL+C to quit

* Restarting with watchdog (windowsapi)

* Debugger is active!

* Debugger PIN: 981-874-726

* Detected change in 'c:\Users\2303A\OneDrive\Desktop\aiac\vscode\1.py', reloading

* Detected change in 'c:\Users\2303A\OneDrive\Desktop\aiac\vscode\1.py', reloading

* Restarting with watchdog (windowsapi)

* Debugger is active!

* Debugger PIN: 981-874-726

Explanation:

- Improved authentication security
- Prevented exposure of credentials
- Enhanced reliability and safety of API

Task 5 – Exception Handling Improvement

Task:

Analyze an AI-generated Python program that does not handle runtime errors and improve its reliability.

Instructions:

- Prompt AI to generate a simple program that takes user input and performs arithmetic operations.
- Identify possible runtime errors (e.g., invalid input, division by zero).
- Modify the program to include proper try-except blocks.
- Provide meaningful error messages for invalid inputs.
- Test the improved version with valid and invalid test cases.

Expected Output:

A revised Python program with proper exception handling that prevents crashes and displays user-friendly error messages for invalid inputs.

Prompt:

Generate a Python program that takes user input and performs arithmetic operations.

Code:

.vscode > 1.py > ...

```
1  '''Generate a Python program that takes user input and performs arithmetic operations.'''
2  # Get user input for the first number
3  num1 = float(input("Enter the first number: "))
4  # Get user input for the second number
5  num2 = float(input("Enter the second number: "))
6  # Get user input for the operation to perform
7  operation = input("Enter the operation to perform (add, subtract, multiply, divide): ")
8
9  # Perform the selected operation
10 if operation == "add":
11     result = num1 + num2
12 elif operation == "subtract":
13     result = num1 - num2
14 elif operation == "multiply":
15     result = num1 * num2
16 elif operation == "divide":
17     if num2 != 0:
18         result = num1 / num2
19     else:
20         result = "Error: Division by zero is not allowed."
21 else:
22     result = "Error: Invalid operation selected."
23
24 # Print the result
25 print("The result is:", result)
26
```

PROBLEMS 4 OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\2303A\OneDrive\Desktop\aiac> & C:\Users\2303A\AppData\Local\Programs\Python\Python313\p
top/aiac/.vscode/1.py
Enter the first number: 12
Enter the second number: 12
Enter the operation to perform (add, subtract, multiply, divide): add
The result is: 24.0
PS C:\Users\2303A\OneDrive\Desktop\aiac> |
```

Explanation:

- Program no longer crashes
- Displays user-friendly error messages
- Improves reliability and user experience